

Homework 4 — The while loop

Due Sunday 11 October, 23:59, on Canvas. The deadline is fixed; late work is not received.

Individual work. One file, two functions and a `main()` — the Homework 3 shape, now with loops. New rule on top of the usual ones: **every loop comes with one sentence that says why it stops** — the quantity that moves toward the exit, and the value where it ends. Write that sentence under the algorithm. A loop without it is incomplete.

What you submit

One file, `A_Matricula_hw4.py`, in the exam format (name comment on line 1, algorithms in `"""`), containing:

1. **`collatz_steps(n)`** — returns how many steps it takes to reach 1 starting from the positive integer n , where one step is: *if n is even, divide it by 2; otherwise replace it by $3n + 1$* . Example: $6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ is **8** steps. No `input()`, no `print()` inside.
2. **`reverse_digits(n)`** — returns the non-negative integer n with its digits reversed: $1234 \rightarrow 4321$. Use `% 10` to take the last digit and `// 10` to drop it; build the answer with `*10 +`. No lists, no strings — the point is the loop.
3. **`main()`** — asks which of the two to run (1 or 2), reads the number, **refuses a number that is not positive (for 1) or is negative (for 2)** with `Input out of range.`, otherwise calls the function and prints the result. The last line of the file calls `main()`.

Plus, as a photo or scan: **the two algorithms** on paper, each with its *why it stops* sentence, and **your shell tests** — what you typed and what came back.

Shell tests — required

```
collatz_steps(1)      0
collatz_steps(6)      8
collatz_steps(7)      16
collatz_steps(27)     111
reverse_digits(1234)   4321
reverse_digits(1200)   21
reverse_digits(7)      7
reverse_digits(0)      0
```

`reverse_digits(1200)` is 21, not 0021 — numbers do not have leading zeros. Say in your algorithm what your loop does with the zeros, and why that is the right answer.

The termination sentence — what I am looking for

- For `reverse_digits`: *" n loses a digit every turn ($n // 10$), so it reaches 0 after as many turns as it has digits."* That is a proof.
- For `collatz_steps`: here is the honest truth — **nobody has proved it stops for every n** . It has been checked for every number up to about 2^{68} , and it has always reached 1, but there is no argument like the one above. So write: *" n does not decrease every turn ($3n+1$ grows), so I cannot name a quantity that always moves toward 1; this loop is believed to stop for all positive integers but it has not been proved."* That sentence,

understood, is worth more than the code. (It is the Collatz conjecture, open since 1937.)

Rubric — published

Both functions return the right values on all eight shell tests (no print, no input inside)	40 %
Algorithms on paper, WHILE and IF lines written out, with the termination sentence for each — including the honest one for Collatz	30 %
main() reads, checks the range with the exact message, calls and prints; file ends with main()	20 %
Shell tests written down as run	10 %

Two points off for each missing technical requirement (name comment, algorithms in "", file name, meaningful names).

If you want more

collatz_steps with a *second* return value: the largest number reached on the way (for 27 it is 9232). Return both: return steps, peak. Then find the n below 100 with the most steps — by hand you cannot; with a loop around your function you can in ten lines.